Terraform 101 - Implementando Bases de Dados do Azure como Código

Wagner Crivelini

SQLSATURDAY
São Paulo

Apresentação: Wagner Crivelini

- Engenheiro de Dados
- Colunista de vários portais de tecnologia com +250 publicações
- Coprodutor do DATABASECAST, 1º podcast brasileiro sobre dados
- Email: wcrivelini@gmail.com

DatabaseCast
www.databasecast.com.br

Spotify

bit.ly/databasecast1

Linkedin

Microsoft | AMD

ENIAC
Centro Universitário de Excelência

DIEGO NOGARE
CONSULTOR EXECUTIVO IA | ML

Pythian

love your data

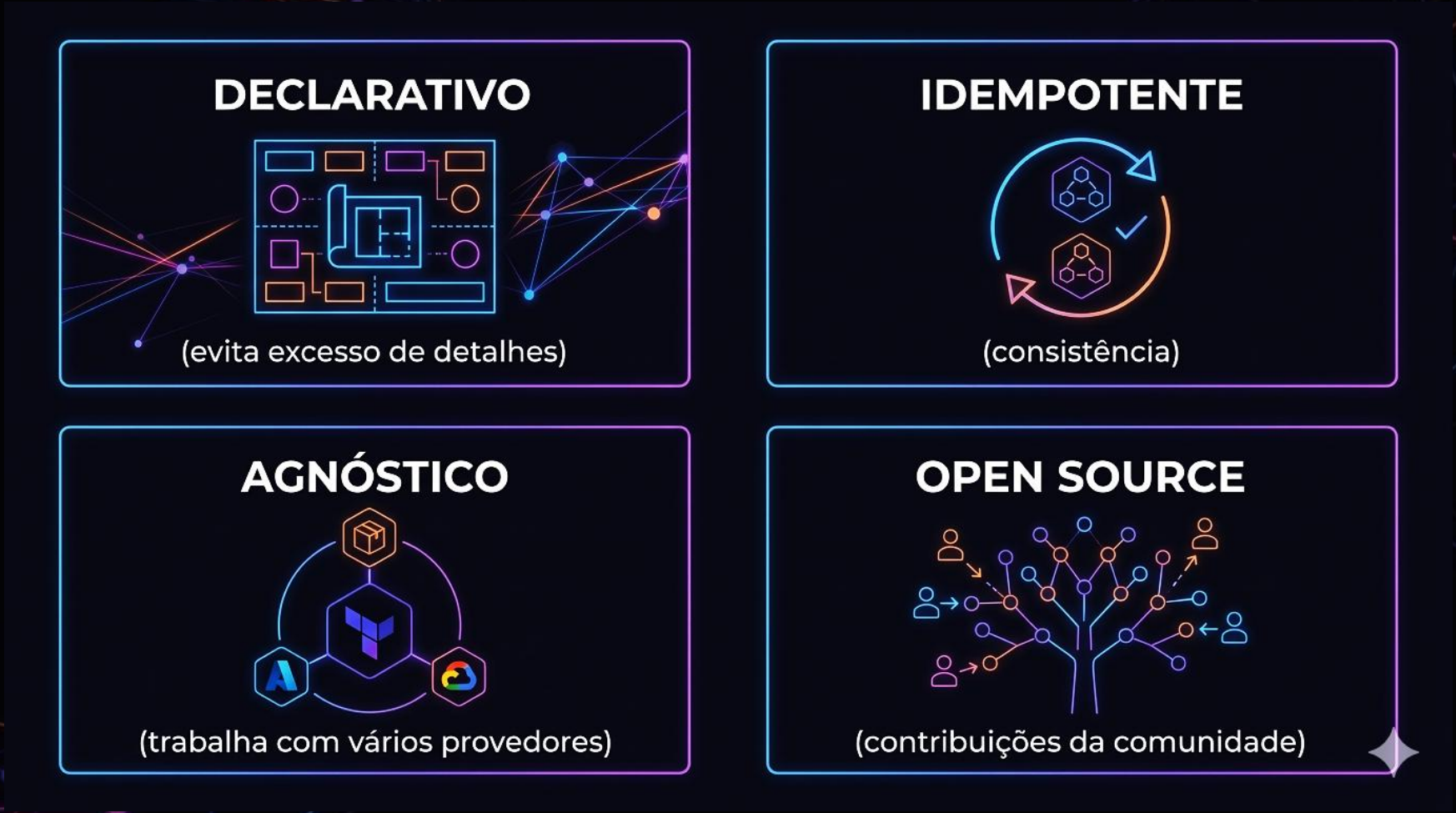
Agenda

1. Introdução
2. Controle de Estado
3. Estratégia
DEMO
Introdução
Agenda
DEMO
Estratégia
RESUMO
4. Múltiplos Recursos
DEMO
Múltiplos Recursos

1. Introdução

Introdução ao Terraform

- Infrastructure as Code (IaC) adotado em larga escala
- Adoção de Terraform (HashiCorp) cresce rapidamente



Introdução – “Getting Started”

Você pode instalar localmente e rodar com VS CODE / CMD
https://www.terraform.io/downloads

Comandos básicos:

```
C:\TEMP\terraform init
C:\TEMP\terraform validate   #seus scripts
C:\TEMP\terraform plan       #plano de execução dos scripts
C:\TEMP\terraform apply      #executa scripts
C:\TEMP\terraform destroy    #caso necessite fazer “rollback”
C:\TEMP\terraform show       #mostra resultado da implementação
```

Introdução – Scripts

Usa Linguagem HCL ou JSON
Permite escolher seu(s) provedor(es) de nuvem:

Para AZURE, use Azure RM

https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs

- Cria/altera recursos apenas informando novos valores
- Usa variáveis ao invés de valores fixos
- Permite criar scripts mais legíveis (script único vs múltiplos scripts)
- Implementação de recursos é feita em paralelo

Introdução – Sintaxe Básica

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
      ##any azurerm version, from 3.0.0
or above
    }}}
```

Scripts começam com a declaração de provedores e suas versões

Identifica parâmetros dos recursos a serem implementados (ex: [https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/resource_group)](https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/resource_group)

```
resource "azurerm_resource_group" "rg" {
  name     = “wagnerresourcegroup2024"
  location = "East Us"
}
```

Introdução – Como Executar Scripts

```
cd <path_to_scripts>
az login
rem to deploy to specific subscription run next command
az account set --subscription <subscriptionID>
terraform init
terraform validate
terraform plan  [-out]  <repositório/arquivoPlan.tfplan>
terraform apply [-auto-approve] <repositório/arquivoPlan.tfplan>
```

DEMO

Introdução

2. Controle de Estado

Controle de Estado

Após primeira execução dos scripts, novos arquivos serão criados

Arquivo TFSTATE identifica o estado de cada recurso após última execução

Antes da próxima execução, Terraform vai checar TFSTATE.

NUNCA PERCA OU ALTERE TFSTATE

Controle de Estado – Como Alterar ou Excluir Recursos

Arquivo TFSTATE é a chave para implementações bem sucedidas
Resultados esperados para próximas execuções de seus scripts:

- Se seus scripts não forem alterados – nada acontece (IDEMPOTENT)
- Se forem alterados valores de variáveis – recursos afetados serão alterados
- Se um recurso for excluído dos scripts – recurso será excluído
- Se o TFSTATE for perdido – TERRAFORM assume que os recursos não existem e tentará recriá-los (gerando mensagem de erro na implantação)

3. Estratégia

Uma Estratégia Melhor

Dividir seu código em múltiplos scripts costuma ser uma ótima opção:

- Melhora gerenciamento (ex: em caso de atualização do AzureRM)
- Melhora escalabilidade (ex: executar implementações em paralelo)
- Melhora reusabilidade (ex: aplicação em ambientes Dev/Tst/Prd)

Uma Estratégia Melhor – Tipos de Scripts

Na prática, é uma boa ideia dividir seu código entre vários scripts
Seu projeto pode incluir (por exemplo):

- [main].tf : script raiz; normalmente provedores e recursos a serem criados.
- variables.tf : onde se define as variáveis, tipos de dados e comentários
- locals.tf : cria expressões baseadas em variáveis (exemplo: criação de “tags”)
- output.tf : coleta valores de atributos durante a criação dos recursos (ex: IDs).
- custom files : arquivos genéricos para customização do seu código
- MÓDULOS : bibliotecas que encapsulam sintaxe de criação de cada recurso

Uma Estratégia Melhor – Porque Usar Módulos?

- Eles isolam a sintaxe correta de criação de cada recursos
- Essa biblioteca deve ser compartilhada com todo time de DEV
- Define um padrão corporativo
- Melhor gerenciamento de upgrades e mudanças de sintaxe dos recursos

Uma Estratégia Melhor – Organizando Scripts Para Usar Módulos

Crie um diretório de MÓDULOS

- modules
  - CosmosDB-Acct
  - CosmosDB-DB
  - Databricks
  - DataLakeGen2
  - DataLakeGen2Account
  - DataLakeGen2FileSystem
  - DataLakeGen2Path
  - Key-Vault
  - ResourceGroup
    - main.tf
    - outputs.tf
    - variables.tf
  - SQL-DB
  - SQL-Server
  - StorageAccount
  - StorageAccountContainer
  - SynapseSQLPool
  - SynapseWorkspace

Crie diretórios de PROJETOS

- example2
  - locals.tf
  - root.tf
  - variables.tf
  - z_ResourceGroup.auto.tfvars
  - z_Tags.auto.tfvars

Uma Estratégia Melhor – Conteúdo dos Scripts: Módulo

modules > ResourceGroup > main.tf

```
#reference https://registry.terraform.io/providers/h
resource "azurerm_resource_group" "ResourceGroup" {
    name                    = var.rg_name
    location                = var.location
    tags                    = var.tags
  }
```

modules > ResourceGroup > outputs.tf

```
output "newResourceGroupName" {
  value = azurerm_resource_group.ResourceGroup.name
}

output "id" {
  value = azurerm_resource_group.ResourceGroup.id
}

#output "tenant_id" {
#  value = azurerm_resource_group.ResourceGroup.tenant_id
#}
```

modules > ResourceGroup > variables.tf

```
variable "rg_name" {
    type  = string
    description = "resource group name"
}

variable "location" {
    type  = string
    description = "code related to Azure Region"
}


variable "tags" {
    type  = map(string)
    description = "tags related to project"
}
```

Uma Estratégia Melhor – Conteúdo dos Scripts: Arquivos do Projeto

example2 > root.tf

```
# Terraform and Azure Provider configuration
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~>3.0"
      ##any azurerm version, from 3.0.0 or above
    }
  }
}
provider "azurerm" {
  features {}
}

module "ResourceGroup1" {
  source      = "../modules/ResourceGroup"
  rg_name     = local.rgname1
  location    = var.location1
  tags        = local.tag
```

example2 > variables.tf

```
# ##################################################
# variables related to RESOURCE GROUP
# ##################################################
variable "resourceIdentifier" {
    type  = string
    description = "preffix to resource group name"
}

variable "location1" {
    type  = string
    description = "code related to Azure Region"
}

variable "shortlocation1" {
    type  = string
    description = "short code related to Azure Region"
}
```

Uma Estratégia Melhor – Conteúdo dos Scripts: TFVARS & Locals

example2 > z_Tags.auto.tfvars

```
company                 = "WWI"
project                 = "platdt"
costcenter              = "xyz"
environment             = "dev"
shortprovider           = "az"
resource_id             = "10"

```

example2 > z_ResourceGroup.auto.tfvars

```

resourcegroup_identifier= "rg"
location                = "southcentralus"
shortlocation           = "ussc"
```

example2 > locals.tf

```

locals {

  rgname1 = format("%s-%s-%s-%s-01"
        , var.shortprovider, var.shortlocation1
        , var.environment, var.resourceIdentifier)

  tag = {
    company       = var.company
    project       = "${var.company}-${var.project}"
    costcenter    = var.costcenter
    environment   = var.environment
  }
}
```

DEMO

Estratégia

4. Múltiplos Recursos

Múltiplos Recursos

- Atenção às convenções de nomenclatura

  https://bit.ly/azurenames

- Esteja atento a nomes únicos
- Considere dependências entre recursos
- Lembre-se que seu script pode combinar o uso de múltiplos provedores

Múltiplos Recursos – Sobre os Nomes Únicos

- No Azure, os recursos precisam ter nomes únicos dentro do seu escopo
- Alguns recursos tem escopo global, portanto é necessário atenção
- Uma alternativa é adicionar um valor aleatório para gerar um nome único (locals.tf)

locals.tf

```
#Random ID for unique naming
resource "random_integer" "rand" {
  min = 000001
  max = 999999
}

locals {
  randomSuffix = "${format("%06s", random_integer.rand.result)}"
  rgname = "${var.rgPreffix}${var.rgName}-${local.randomSuffix}"
}
```

Múltiplos Recursos – Dependências Entre Recursos

Muitos recursos dependem de um recurso “pai”

Nesses casos, esteja atento a 2 pontos principais:

O recurso “filho” deve ser implementado DEPOIS do “pai”.

Garanta que o “pai” forneça as informações necessárias para definição do recurso “filho” (outputs.tf)

modules > SynapseWorkspace > outputs.tf

```
output "name" {
  description = "The name of the resoruce created."
  value       = azurerm_synapse_workspace.synapsewokspace.name
}

output "id" {
  description = "The id of the resource created."
  value       = azurerm_synapse_workspace.synapsewokspace.id
}
```

example3 > root.tf

```
module "synapsesqlpool" {
  source                                  = "../modules/SynapseSQLPool"
  synsqlpool_name                         = var.synsqlpool_name
  synsqlpool_wrkspcid                     = module.synapseworkspace.id
  synsqlpool_sku                          = var.synsqlpool_sku
  synsqlpool_mode                         = var.synsqlpool_mode

  depends_on = [module.synapseworkspace]
}
```

Múltiplos Recursos – Mensagens Exibidas

Terraform apresentação mensagens de status durante a implementação (CMD)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [3m50s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m0s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m10s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m20s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m30s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Still creating... [4m40s elapsed]
module.synapseworkspace.azurerm_synapse_workspace.synapsewokspace: Creation complete after 4m49s [id=/subscriptions/56508de2-630e-4d64-a40f-45bfa674520b/resourceG
roups/az-ussc-dev-rg-563645/providers/Microsoft.Synapse/workspaces/az-ussc-dev-syn-563645]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Creating...
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m0s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [1m50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m0s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m10s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m20s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m30s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m40s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Still creating... [2m50s elapsed]
module.synapsesqlpool.azurerm_synapse_sql_pool.synapsesqlpool: Creation complete after 2m54s [id=/subscriptions/56508de2-630e-4d64-a40f-45bfa674520b/resourceGroup
s/az-ussc-dev-rg-563645/providers/Microsoft.Synapse/workspaces/az-ussc-dev-syn-563645/sqlPools/whiterabbit]

Apply complete! Resources: 17 added, 0 changed, 0 destroyed.
PS C:\temp\Terraform101\example3>
```

DEMO

Múltiplos Recursos

RESUMO

RESUMO

- Terraform permite criar virtualmente qualquer recurso do Azure
- Linguagem do Terraform é bastante versátil e poderosa
- Princípios apresentados aqui também se aplicam a outros provedores
- Existem vários projetos open source que melhoram o Terraform

REFERÊNCIAS

GitHub – All Files Presented in this Workshop:
https://github.com/wcrivelini/articles/tree/main/Azure_Terraform

Database Deployment with Terraform - The Basics:
https://www.sqlservercentral.com/articles/database-deployment-with-terraform-the-basics

Database Deployment with Terraform – Modules:
https://www.sqlservercentral.com/articles/database-deployment-with-terraform-modules

DatabaseCast
www.databasecast.com.br

Spotify

bit.ly/databasecast1

Obrigado

Wagner Crivelini

Linkedin

Email: wcrivelini@gmail.com